The Replay Lattice: A Type System for Floating-Point Reproducibility

The foundation of a determinism-safe language

M. A. J. Haddon
michael@politepixels.io
PolitePixels
Saint Helier, Jersey, Channel Islands

Abstract

Add the same floating-point numbers in a different order, and the last bits of the sum can change. A parallel reduction groups its input by worker count, and one program can then return different bits on different worker counts. Hence, a promise of bit-for-bit reproducibility depends on an unannounced schedule.

A floating-point value's determinism is a *demand carried on the value*, one that holds only if every operation that ever touched the value obeyed it. Almost every mechanism the field has built to govern reordering is instead a *grant attached to a site*, a licence to reorder at one place in the program text. A grant is true where it is written. A demand's truth is settled upstream, by the value's history, which no single site can see. We present the Replay Lattice: three schedule-class contracts, `exact` below `replay` below `approx`, carried on every value of a pure dataflow array language, joined over the dependency path, checked at each declaration, and discharged into backend lowering obligations.

Two theorems come with the lattice. Soundness is a two-run induction over the dataflow graph, mechanised in Coq. The poisoning law says one free reordering taints everything downstream, and its proof is one line on top of the join.

We also audit our own evidence, finding ten occasions on which a passing check of ours had proved nothing, and repairing every one. The usual figure, no rejections over an unannotated corpus, holds by construction for a demand-only checker, which is why we do not offer it as evidence.

Finally, the artefact ships the checker and its nineteen must-fail controls. The result is a language in which *these bits must not depend on the schedule* is a sentence the compiler checks.

CCS Concepts

• **Theory of computation** → **Type structures**; • **Software and its engineering** → *Functional languages*; *Parallel programming languages*; *Domain specific languages*; *Compilers*; *Formal software verification*; • **Mathematics of computing** → Numerical analysis.

Keywords

determinism types, type qualifiers, floating-point determinism, floating-point reproducibility, schedule nondeterminism, reassociation, 2-safety hyperproperties, dataflow array languages, blame

1 Introduction

Add `0.1`, `0.2` and `0.3` as doubles. Grouped left to right you get `0.6000000000000001`; grouped as `0.1 + (0.2 + 0.3)`, you get `0.6`. However, the programmer does not choose which grouping a parallel program performs. A parallel reduction chunks its input by worker count, which is whatever the deployment provides.

This is more than a rounding curiosity. When a phylogenetics group reran 10,179 published datasets with only the parallelism changed, 31% of them came back with different answers [38]. §7.5 replays that failure inside the artefact, with the repair beside it. The same effect is why a bit-exact regression test breaks under a new thread count, and why, in machine-learning inference, one prompt can yield different tokens depending on what else the server was batching [17].

The problem is concrete for us. We maintain a data-parallel array language whose purpose is reproducibility. While making it production-ready we hit an optimiser bug in its compiler, and that incident forced the discipline of carrying the demand through lowering. The program it broke, in four lines:

```
x := indices(count: 100000.0)
a := sum(x)
replay b := sum(x)
return a - b
```

Our optimiser merged the two folds, because they were identical in every property it modelled, and silently pinned a value nobody had asked to pin. We expand upon the fix in §6.3. The language is our own and unreleased, which is why it is not named; no claim below rests on a measurement of it. We implement the checker for lanelang, a demonstration language built for this paper and released in full as a public artefact (Appendix A). The lattice itself is tied to neither language, because all it reads is a dependence graph and a table of operation classes.

Almost every mechanism the field offers is a *grant attached to a site*, a licence saying *you may reorder here*. Table 1 is a census of these mechanisms. Six of its rows state a demand, and five of the six attach to something other than a value, among them the per-call determinism level of CCCL, NVIDIA's CUDA Core Compute Libraries [32]. The exception is GLSL's `precise`, which is a demand on a value, and the closest row to what this paper builds.

Reproducibility is not a grant. "These bits must not depend on the schedule" is a *demand attached to a value*, and the site that writes it cannot check it, because the folds that decide those bits sit upstream, in code the site cannot see, and which schedule they ran under was the deployment's choice. The join is what makes the demand checkable. This subject, alone amongst its neighbours, admits no escape downstream of a free fold, because the rounding has already happened, and no later operation recovers the discarded low bits.

Our evidence for the gap is where the existing mechanisms stop, and CCCL's stopping point is visible in its source. In particular, level resolution consults only the current call so that a `not_guaranteed` result may feed a `run_to_run` call, the weakest level feeding a stronger one, with no diagnostic anywhere [32]. The scheme answers the local question well and cannot ask the composition question at all, because no determinism level outlives the call. Check L1 runs that as an experiment inside the vendor's own compiler, where both of the diagnostics the library promises fire on the two local misuses, and the composition draws neither.

What this is. This work is a type system for one property of one value, not a compiler, not a numerical method, and not a claim that any one schedule is the right one. Every number is asserted by a numbered check a reader can run and listed against its claim in the artefact (Appendix A). The exceptions are flagged where they occur: a few measurements come from one machine with no check behind them, and claims about the production compiler are observations a reader cannot rerun. §2 weighs the prior art on ordering, and §8 covers the rest.

Contributions.

1. The reframing of a floating-point schedule class as a demand carried on a value, and the type system it forces: three classes, a join over the dependency path, an inference pass that strengthens a producer only where nothing outside the demand reads it, and per-class backend lowering obligations (§3, §4).
2. A reading of the class at lowering time that leaves fusion, tiling, and every dependence-preserving reorder free while forbidding reduction splitting and the merging of twin reductions under value numbering (§6.3).
3. A mechanised soundness theorem for a schedule-indexed semantics, proved by two-run induction and checked in Coq with the poisoning law beside it, conditional on two stated assumptions, Assumptions 1 and 2, made concrete as a checked table (§3.5, §5).
4. An evaluation that reports its positive evidence with figures, then audits itself, finding and repairing ten occasions on which a passing check of ours had proved nothing (§7).

2 Grants and demands

Try to write the requirement *these bits do not depend on the schedule* in any system in Table 1. The first thing to look for is a promise that sits on the value itself, not on some line of code the value once passed through. Second, it has to be a requirement, not a permission to reorder. Third, somebody has to actually check it, over everything that ever touched the value. Fourth, it has to become an obligation on whoever picks the schedule. Go and look.

2.1 Determinism-safe languages

Every language that computes in floating point has a schedule: an order in which its reductions combine, set by the width at which they block or by a worker count that may vary between runs. However, almost none enforces anything about it. Reproducibility is lost in the gap between knowing and enforcing.

This paper borrows type safety as its framing, the property Milner proved, that well-typed expressions cannot "go wrong" [28]. Every JavaScript value knows its own type at runtime, and yet `'1' + 1` becomes `'11'`: the language coerces instead of refusing. Knowing a type and enforcing one are different things. Determinism non-safety can be defined in the same way, just on the topic of the schedule. If you add a schedule-pinned value to a value that is not, and the language coerces them together, then we can call it determinism-unsafe. Therefore, the remedy is the same as the type-safe one, which is refusal. We also place it at the declaration. What differs is that we must check everything that ever touched the value, not just the expression at that line.

With that in mind, we say a language is *determinism-safe* when five criteria hold:

(i) Every value carries a schedule-determinism class.
(ii) The class is a function of the value's whole history, whereas a per-site knob is a function of the site where it is written.
(iii) A declaration states a required class and is checked against the computed one.
(iv) Violation is *rejection*, not advice, not a warning, not an inhibited optimisation.
(v) Acceptance discharges into a stated obligation on whoever picks the schedule.

Table 1: Every mechanism that governs reorder freedom: what it attaches to, and where its truth is settled.

System	The knob	Attached to	Direction	Enforced?	Discharged into
LLVM/MLIR [45, 46]	`reassoc`, `contract`	an instruction or operation	grant, local	no	the optimiser
Icing `opt:` [7]	a scope in the value grammar	an expression	grant, local	verified	a verified compiler
SPIR-V [44]	the `FPFastMathMode` decoration	an instruction	grant, local	no	the driver
Halide `rfactor` [39]	a proved associativity	a schedule directive	grant, local	in the reals	the schedule
Futhark `reduce` [18]	a promise	an operator	grant, local	no	the compiler
IEEE 754 [19]	a reproducibility attribute	a lexical block	**demand**, in block	by the language	the implementation
OpenMP [33]	the `reduction` clause	a construct	grant, local	no	the runtime
CCCL `require` [32]	a level	one call	**demand, not local**	at that call only	a kernel choice
oneTBB reduce [48]	a split/join fixed per partitioner	one call	**demand, not local**	unchecked	a library kernel
ML inference [17]	an order fixed by extent	one kernel	**demand, not local**	unchecked	an implementation
LS-DYNA [24]	the consistency keyword	a run	grant, local	no	the solver
GLSL `precise` [43]	a qualifier	**a value**, and its producers	**demand**, in function	no computed class	the shader compiler
`MPI_REDUCE` [27]	an implicit reorder licence	a collective call	grant, local	no	MPI implementation
Kernel contracts [49]	a three-level class	an operation class	**demand, not local**	by repeated invocation	a conformance verdict
This work	a lane (§3)	**a value**	**demand, not local**	**whole path**	**backend obligations**

Only the subject of these five criteria is new. Substitute a secrecy label for the class and the emitted instruction sequence for the schedule, and they describe a checked information-flow system, which is where the shape comes from. Volpano, Irvine and Smith gave the first sound one in 1996, and its soundness theorem is already the two-run induction this paper's proof uses [50]. Under that substitution, FaCT satisfies all five for constant-time execution [10]. We claim neither the shape nor the demonstration method. We claim the subject and what the subject forces.

Run Table 1 against these five criteria, and every row above ours fails. A grant states no class at all. A demand attached to a site states one, and the site cannot see the value's history. GLSL's `precise` names one on the value and never computes it. A *determinism-safe language* is one where all five criteria hold at once.

Determinism-safe does not mean *deterministic*, since a language can produce the same bits every run on one machine and be determinism-unsafe, because no rule prevents a future schedule from changing them.

2.2 Why a demand is not settled locally

Two kitchen labels make the point. *May contain nuts* is settled at the dish that carries it. *Contains no nuts* is a claim about every ingredient that ever entered the pot, one nut anywhere falsifies it, and no later stirring takes it back out.

A grant is the first kind of sentence, existential over one site: *this operation may have been reordered.* Because its truth is settled locally, a second grant elsewhere cannot falsify it.

A demand is the second kind, universal over a history: *no operation that contributed to this value can be reordered in a way that depends on the schedule.* Its truth quantifies over the value's ingredients, every input that fed it, directly or through others. Adding one operation anywhere in that history can falsify it, and no later operation can repair it, because the rounding it performed is already in the bits.

This one-way loss is the poisoning law, and it is why a per-call API is not simply inconvenient but *inexpressive*. When CCCL's `run_to_run` reduction receives a pointer, the pointer is an unlabelled jar: it carries no history.

By contrast, GLSL has carried a demand on a *value* since 2010, where `precise` forbids reassociation and contraction, and propagates backward to "operations contributing to a variable's value", although only within one function [43]. The demand is on the value, and the reasoning is still per-site.

2.3 The structural argument

The same point has a structural version. Calling a value schedule-invariant compares two runs of the program, one under schedule `s` and one under `s'`, bit for bit, and the literature calls such pair-of-runs properties 2-safety [12]. A check attached to one call sits inside a single run and cannot see the other.

The join is what makes a 2-safety demand checkable at all. It summarises the entire dependency path into one lattice element that travels with the value, which means a local comparison at the declaration site suffices. One fold left free upstream joins in as `approx`, and every expression its result touches inherits that. A declaration demanding `replay` then fails locally, no matter how far away the fold was. The move is information-flow control [50], made for thread scheduling by observational-determinism type systems [40, 51], whose subject is interleaving on a shared location. A reduction lowered here has no interleaving to observe, because each worker owns its accumulator and the combine reads them in a fixed order.

2.4 The standard already drew this line

Clause 5 of IEEE 754 requires each basic operation to be correctly rounded, and both groupings in the opening example obey it. The bits differ anyway, because nothing varies between them except the bracketing, which is why per-operation correctness cannot carry reproducibility. The standard draws the same line itself: reductions sit in a clause of their own, 9.4, which says implementations "may associate in any order or evaluate in any wider format" [19]. And the clause titled "Reproducible floating-point results" has never, since it arrived in 2008, covered 9.4, which means a reordered sum can change its bits and break no clause.

2.5 Where checked guarantees stop

Several checked systems come close to the five criteria. This section asks one question of each: where does the checked guarantee stop? The answer is always a deliberate escape, a marked place where the programmer steps outside the check. DPJ, Deterministic Parallel Java [9], checks memory effects and takes the programmer's word for commutativity, as its authors state plainly: commutativity "is not checked by the compiler". EnerJ, a type system for approximate computing [36], keeps such values out of precise code, except where the programmer *endorses* one back in.

The schedule is the subject on which no escape can be built. Both escapes above hand the programmer something to certify, EnerJ a bounded loss in a domain built to absorb one, a pixel a shade off, and DPJ an unchecked annotation in the source, where a reader can grep for it and weigh the assumption. A reordered sum leaves neither. The low bits each rounding discarded are gone; the bits that remain are whatever the schedule made them, and the poisoning law proves the direction: a value that reached `approx` cannot be brought back. So where EnerJ and DPJ may summarise a history, certify it and drop it, this design has to carry the history on the value. The class therefore rides the value and joins along the dependency path, and the five criteria are forced, being what checking looks like when the history cannot be dropped.

The Replay Lattice's one escape is where bytes enter from outside the program. An example is an input file. Knowing whether it will hold the same bytes run to run is a fact about a deployment instead of arithmetic, which means no checker can decide it. A blessing (§4.6) lets a programmer take that obligation on in writing, such as a checksum the run verifies.

3 The Replay Lattice as a type system

In lanelang, the demonstration language, a program is a dataflow graph of bulk array operations over `double`. The language is pure, first-order, total and single-assignment, and the graph is acyclic apart from one carried edge per recurrence, a loop-carried dependency in compiler terms. *The schedule is a parameter of the semantics,*

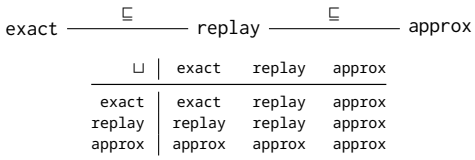
not an effect of the program. The proofs reason about which bracketing a reduction used, and interleaving does not come up.

3.1 The three classes

The lattice has three points, the classes, and each is defined by the set of schedule pairs across which a value's bits must agree. The class a value carries is its *lane* (not the vector sense). A lane is this system's analogue of an information-flow label. All of this lives at compile time, and no class is carried or checked at runtime.

- **`exact`**: the value equals the serial reference oracle's result under the fixed build (one toolchain and ABI). Discharged either by exact-integer arithmetic, where every schedule agrees because the elements are integers whose magnitudes sum below 2^{53}, where addition is exact and associative, or by an oracle-order strict fold, which fixes the order and costs all reduction parallelism.
- **`replay`**: the bits agree across every schedule in S (§5.1), whatever the worker count or traversal order; the host compiler, vector width and ABI sit outside `S`. This artefact discharges it with a fixed-tree schedule whose shape is a function of the input shape alone. It is parallel, and the name is the point, because you can rerun the program under a different worker count and the bits replay exactly, which is what an audit needs.
- **`approx`**: no constraint. Reorder freely, maximum parallelism, schedule-dependent bits. Its kernel is the *free kernel*, which may reassociate.

`exact` does not mean correctly-rounded. It is a schedule claim, equality with a named reference, and an oracle-order float fold accumulates rounding error like another; it simply accumulates the same error every time. The class asks for agreement across every schedule and equality with the reference, and an error-reducing kernel (Kahan [20], Neumaier [31], a long accumulator [21]) is neither required nor sufficient.



⊔	exact	replay	approx
exact	exact	replay	approx
replay	replay	replay	approx
approx	approx	approx	approx

Figure 1: The lattice and its join.

For a floating-point reduction, the costs assign the roles. `exact` is the check, serial and therefore off the shipped path. `replay` is what ships. `approx` is the default, where everything else stays.

Why `approx` is the default. Defaulting to `replay` would change every unannotated program's bits on the day the flag flips, and a discipline adopted only by a rewrite does not get adopted.

3.2 The lattice and the join

The order is a result, not a choice. Each class promises which runs must agree, formalised as a set of schedule pairs. `approx` promises nothing. `replay` promises that the bits come out the same under any schedule in S. `exact` promises that, and that the bits also match the serial reference. Each promise strictly contains the weaker one's, which means the three classes form a chain (Figure 1), and the join is `max`. Theorem 2 is then one line, because a chain has a single top. Read $\sqsubseteq$ as substitutability, meaning a stronger guarantee may flow wherever a weaker one is demanded, and the reverse is refused.

3.3 Where a lane attaches

A lane is demanded with a qualifier, written bare where a value gets its name, and the qualifier has three positions, one line each:

```
replay energy := sum(x)    // on a binding
func f(replay v Float)     // on a parameter
func f() replay Float      // in a return type
```

Three positions are enough, because the way to demand part of an expression is to name that part with a binding. Naming is free at runtime, since equal subexpressions already share one node in the interned graph, as under value numbering. Demanding adds one node to that graph, the node that carries the demand, and may end sharing, because a fold under a demand no longer merges with an identical free one, which is the opening incident forbidden by construction. Two properties of a qualifier are load-bearing, and an implementation that gets either wrong changes which programs are accepted:

1. A qualifier binds to the value its binding names, and to nothing else. Write `replay t := sum(x*y)` and then `t + other`, and the demand covers the fold while `other` is left alone.
2. A qualifier covers everything written on its binding's right-hand side, the way a pragma or a lexical scope covers the code under it, which is why the rules call the demanded lane in force the *ambient lane*. For example, in `replay t := sum(x*y)` the multiply is inside the demand, not only the sum. The nodes that feed a value, directly or through others, are its *cone*, and when demands nest, the inner one wins for its own part of the cone.

A lane in a function's return type is a promise about the value it hands back, and the body is checked against the promise rather than covered by it. Folds in the body start free, and an inference pass, which we call healing, strengthens the ones private to the demand (§4.4). Every declared function is lowered and checked on its own, including one without callers, and an argument is checked in the caller's scope against the lane on the parameter it fills.

3.4 The grammar and the typing rules

Figure 2 is the language the checker parses, with $\ell^?$ an optional lane and $\overline{z}$ a sequence. The figure is drawn from the implementation, not an idealisation of it. A call's argument labels and arities are checked later, at resolution. The three lane words and `ungrounded` are reserved, which stops a variable from shadowing them and a mistyped demand from silently becoming one.

The rules do three jobs. They compute each value's lane and check it wherever a demand is stated. They decide whether the program fixes a value's bytes. And they decide whether every leaf it does not fix carries a blessing, introduced later. Three judgements carry the jobs: typing, over the grammar, assigns each expression a type and a lane; grounding and *waiver* answer the other two over the resolved graph, where a bound name stands for the node its binding

built. Figure 3 prints the rules whose shape is a decision, beside the prose that reads them. The four structural rules are routine: the two base cases, the join at each elementwise node, and subsumption along the order.

$$
\begin{array}{rl}
\tau & ::= \texttt{Float} \mid \texttt{Column[Float]} \\
\ell & ::= \texttt{exact} \mid \texttt{replay} \mid \texttt{approx} \\
q & ::= \texttt{ungrounded}^{?}\ \ell^{?} \\
\beta & ::= \texttt{@bless} \mid \texttt{@bless}(n) \\
op & ::= \texttt{+} \mid \texttt{-} \mid \texttt{*} \mid \texttt{/} \qquad red ::= \texttt{sum} \mid \texttt{isum} \qquad scn ::= \texttt{scan} \\
gen & ::= \texttt{indices} \mid \texttt{integers} \qquad src ::= \texttt{source} \\
\lambda & ::= (x,\, x) \Rightarrow e \mid (x,\, x) \Rightarrow \{\, \overline{b}\ \texttt{return}\ e\,\} \\
a & ::= e \mid x\texttt{:}\ e \mid \texttt{order: oracle} \mid \texttt{with:}\ \lambda \\
e & ::= n \mid x \mid (e) \mid e\ op\ e \mid red(\overline{a}) \mid scn(\overline{a}) \\
 & \quad \mid\ \ gen(a) \mid src(a) \mid \texttt{choose}(\overline{a}) \mid \texttt{iterate}(\overline{a}) \mid f(\overline{a}) \\
b & ::= \beta^{?}\ \ell^{?}\ x\ \texttt{:=}\ e \\
p & ::= \ell^{?}\ x\,\tau \\
d & ::= \texttt{func}\ f(\overline{p})\ q\ \tau \\
 & \qquad \{\, \overline{b}\ \texttt{return}\ e\,\} \\
P & ::= \texttt{package}\ x\,\overline{d}
\end{array}
$$

Figure 2: The grammar.

The grammar's operations, in one clause each:

- `indices` generates a column of the given length
- `source` models an external input, sized by its count
- `sum` and `isum` reduce a column, `isum` in integer arithmetic
- `choose` is a masked select, and it reads both columns
- `iterate` is a recurrence, running its `with:` step `n` times

A count is written as a float because the demonstration language has one numeric type. Worked in full over one program:

```
func main() replay Float {
    // exact                          | ungrounded | waived, blessed
    @bless raw := source(count: 8.0)
    // replay, healed                 | ungrounded | waived
    s := sum(raw)
    // replay, healed                 | ungrounded | waived
    total := sum(raw * s)
    return total                      // the declared lane, met
}
```

The three comment columns are the three judgements. The return lane demands `replay` of everything the body computes; `s` and `total` are free folds private to that demand, so healing strengthens both; `raw` is a column the program does not fix, so no node above it is grounded, and the blessing §4.6 gives takes that obligation on in writing. Read `s` once more, as `return total + s` with the demand moved onto `total`, and healing may no longer strengthen the fold; that twin is refused with blame at both ends.

The four judgement forms, left to right, are typing's expression form, typing's binding form, grounding, and waiver:

$$\Gamma;\ \ell_a \vdash e : \tau^{\ell} \qquad \Gamma;\ \ell_a \vdash_{bind} b \dashv \Gamma' \qquad \Gamma \vdash e \Downarrow g \qquad \Gamma \vdash e \Downarrow_b w$$

Read the first as: under environment Γ and ambient lane ℓ_a, expression `e` has type τ and computed lane ℓ. The binding form adds the bound name to the environment at that lane.

T-Fun's body is typed under no ambient lane at all, and that is the promise. A declared return lane does not descend; the body's folds are typed free, and the lane is compared against what they reach. The parameters enter the environment at the join identity

$$\frac{\Gamma;\ \ell_d \vdash e : \tau^{\ell} \qquad \ell \sqsubseteq \ell_d \qquad \Gamma \vdash e \Downarrow_b w \qquad c = \textbf{waived} \text{ if } \beta,\ w \text{ otherwise}}{\Gamma;\ \ell_a \vdash_{bind} \beta^{?}\ \ell_d\ x\ \texttt{:=}\ e \dashv \Gamma, x\!:\!\tau_c^{\ell}}\ \textsc{T-Bind}$$

$$\frac{\Gamma;\ \ell_a \vdash e : \tau^{\ell} \qquad \Gamma \vdash e \Downarrow_b w \qquad c = \textbf{waived} \text{ if } \beta,\ w \text{ otherwise}}{\Gamma;\ \ell_a \vdash_{bind} \beta^{?}\ x\ \texttt{:=}\ e \dashv \Gamma, x\!:\!\tau_c^{\ell}}\ \textsc{T-BindFree}$$

$$\frac{\Gamma;\ \ell_a \vdash e : \tau^{\ell} \qquad k = \mathrm{kernel}(red, \ell_a, o) \qquad \ell_k = \mathrm{lane}(\mathrm{bits}(k))}{\Gamma;\ \ell_a \vdash red(e,\ (\texttt{order: } o)^{?}) : \tau^{\ell_k \sqcup \ell}}\ \textsc{T-Fold}$$

$$\frac{\Gamma;\ \ell_a \vdash e : \tau^{\ell} \qquad k = \mathrm{kernel}(scn, \ell_a) \qquad \ell_k = \mathrm{lane}(\mathrm{bits}(k))}{\Gamma;\ \ell_a \vdash scn(e) : \tau^{\ell_k \sqcup \ell}}\ \textsc{T-Scan}$$

$$\frac{\begin{array}{c} f \text{ declares } \ell_i^{?}\ x_i\!:\!\tau_i \text{ and returns } \ell_f^{?}\,\tau \\ \Gamma;\ \ell_a \vdash a_i : \tau_i^{\ell_i'} \qquad \ell_i' \sqsubseteq \ell_i \text{ wherever } \ell_i \text{ is written} \\ \Gamma;\ \ell_a \vdash inline(f, \overline{a}) : \tau^{\ell} \qquad \ell \sqsubseteq \ell_f \text{ wherever } \ell_f \text{ is written} \end{array}}{\Gamma;\ \ell_a \vdash f(a_1 \ldots a_k) : \tau^{\ell}}\ \textsc{T-Call}$$

$$\frac{\begin{array}{c} \Gamma_0 = \overline{x\!:\!\tau^{\texttt{exact}}} \qquad \Gamma_{i-1};\ \emptyset \vdash_{bind} b_i \dashv \Gamma_i \quad (i = 1 \ldots n) \\ \Gamma_n;\ \emptyset \vdash e : \tau^{\ell} \qquad \ell \sqsubseteq \ell_f \text{ wherever } \ell_f \text{ is written} \end{array}}{\vdash \texttt{func}\ f(\overline{p})\ \ell_f^{?}\,\tau\,\{\, b_1 \ldots b_n \qquad \texttt{return}\ e\,\}\ \textbf{ok}}\ \textsc{T-Fun}$$

Figure 3: The binding, kernel and boundary rules.

(`exact`, the bottom of the chain), because a parameter's lane, like its grounding, is the caller's obligation rather than a fact the body may assume. The lane a parameter declares binds the caller, discharged by T-Call at each call site, whose `inline` premise retypes the inlined body under the call's actual lanes, so a promise the body keeps only for stronger arguments is refused at the call that supplies weaker ones. The price is modularity: a function has no principal lane signature short of lane polymorphism.

T-Bind is the rule that checks. It types the right-hand side under the demanded lane, requires the computed lane to be at or below the demand, and binds the name at the computed lane. T-BindFree is the same rule, but without the comparison. Both read a blessing wherever one is lawfully written.

T-Fold carries the subject, in two steps. The ambient lane and the order dial pick which kernel the reduction lowers to; the kernel's row (§3.5) supplies the class. A demand reaches the first step and never the second, the discharge invariant written as a premise. Because join is `max`, an `approx` fold *overrides* its inputs, while a `replay` fold merely *caps*.

The order dial is per-fold, spelled `order: oracle` inside the fold it instructs. It pins that call's fold order and states no demand.

A recurrence is the ordinary call it is written as, its lane a least fixed point over the carried edge, computed by the checker's fixed-point iteration. A step function is reachable only as the value of a `with:` label, which keeps the language first-order by construction.

Grounding and waiver are judgements of their own, in that neither appears in a superscript and no rule orders them against a lane. Keeping them apart stops a promise about one input from being recorded as a fact about the program. A leaf is grounded exactly when the program fixes its value, and every other node, §4.5's wrapper aside, exactly when all its reads are, including the carried edge. Waiver adds one case: a leaf also counts as waived when it is blessed. Acceptance requires both verdicts, the lane comparison and the waiver bit, with no join between them.

A parameter is grounded by assumption, because its value is the caller's obligation (T-Call).

Table 2: The operation-class table: each kernel's class, decided by what its output bits depend on.

Kernel	Its bits are a function of	Class	Witnessing check
elementwise arithmetic	nothing of its own: no reassociable combination, and the host compiler sits outside `S` (§5.1)	neutral	**L2** (combination half)
data movement, literals	nothing of its own: no combination at all	neutral	**L2**
external source	the world outside the program, not the schedule; a demand over it is refused by the grounding judgement rather than weakened	neutral, **ungrounded**	L3, CC1, **L2**
carried-state recurrence	nothing of its own: strictly sequential, so it has exactly one schedule	neutral	L3, L4, **L2**
fixed-tree reduction	(data, shape)	`replay`	L4, **L2**, CC2
fixed-block scan	(data, shape): each block scans serially and a serial base pass combines the totals	`replay`	L3, L4, CC3, **L2**
oracle-order strict fold	data alone, **and** equal to the serial reference by construction	`exact`	L4 (reference comparison), CC4, **L2**
exact-integer reduction	data alone, **and** equal to the serial reference: while the column holds integers whose absolute values sum below 2^{53}, addition is exact, so the free order and the serial order agree	`exact`	L3, L4, **L2** (chunking)
free reduction, free scan	the schedule: licensed to reassociate	`approx`	L4's unpinned cells, **L2**
parallel scatter	**UNCHECKED**: no parallel scatter exists here; named again in §5.5	`approx`	none
transcendental functions	**UNCHECKED**: lanelang pins no cross-platform math library	`approx`	none

3.5 How each operation's class is decided

So far, every rule moves classes around, and none of them decides one. A class is decided once per kernel, before any program exists, by one question: what are the kernel's output bits a function of? If the schedule is anywhere in the answer, the class is `approx`. If the bits are a function of the data and its shape alone, the class is `replay`. If they are a function of the data alone and equal to the serial reference under the fixed build, the class is `exact`. A kernel that combines nothing is written *neutral*, and its lane is the join's identity, `exact`, so it contributes nothing.

Table 2 is that question asked eleven times, once per row. The typing rules read its answers, written `bits(k)`, and never define them. Where an operation has more than one kernel, as a reduction does, the demand's ambient lane and the per-fold order dial pick which kernel the fold lowers to, and that kernel's row decides.

That leaves one question: why trust the table? The rows enter as assumptions, because what a kernel does with the schedule is an arithmetic fact typing cannot derive. §5 is conditional on them, and they are proved of a model evaluator, the kernels restated in Coq, not of the shipped C. The integer row also guards its own domain: an element, or the column's magnitude sum, outside the exact range fails the evaluation rather than returning a rounded answer under a class it has not earned.

What the checker takes from the table is `class(o)` alone. Everything else the checker computes itself.

4 Propagation, checking and blame

The site where a demand is written cannot settle it, so the checker brings the history to the site, folding everything upstream into one class that rides the value, and the declaration then needs only a local comparison. Section 4 gives that checker, which decides every demand and, on refusal, names where the guarantee was lost.

4.1 The algorithm

The checker is a single forward pass over the graph, in dependency order, computing at each node:

$$\texttt{computed}(v) = \texttt{join}(\texttt{op_class}(v), \texttt{computed}(in_1), \ldots, \texttt{computed}(in_k))$$

A recurrence's carried value joins as one more input. At every node that states a demand, `computed(v)` must lie at or below `declared(v)` in the lattice's order, or it is an error; nodes are constructed in dependency order, which is what lets one left-to-right sweep suffice.

Except across a recurrence. A recurrence sequentially carries its operand state across iterations, which a single sweep cannot track. The sweep therefore runs to a fixed point, repeating until a full pass changes nothing. For a recurrence nested `d` deep, the sweep settles within `d + 2` passes, `d` for the news to cross the nested carried edges, one to climb, one to validate it is stable.

The pass computes the declarative type. The typing rules say which programs should be accepted. The checker is the code that actually decides, and if the two could disagree, the proofs would certify a checker nobody ships. This result says they cannot: *for every expression* `e`, *`computed(e)` is the least class the rules of §3.4 can derive for it.* The statement has one half per verdict. Because the sweep's answer is always derivable, every acceptance it grants is one the rules grant. Because it is the least derivable, every rejection it issues is one the rules issue.

$$\frac{\Gamma;\ \ell_a \vdash e\ :\ \tau^{\ell} \qquad \ell \sqsubseteq \ell'}{\Gamma;\ \ell_a \vdash e\ :\ \tau^{\ell'}}\ \textsc{T-Sub}$$

T-Sub lets a derivation answer any class above the join, so a sweep that returned anything higher would reject a program the rules accept. Hence the checker reaches the lane verdict the rules derive at every binding of the healed graph. Healing runs before the check, leaving this result and §5.3 quantifying over the same graph, and while the derivable half is stated over the acyclic fragment, the fixed-point presentation carries it over recurrence.

4.2 Monotone acceptance

The checker enforces only what a programmer wrote, and that is provable.

A demand-only checker never rejects a program that states no demand.

Immediate by cases on the binding rules. A program stating no demand writes no qualifier, and every binding in it is then typed by T-BindFree, which states no at-or-below condition. T-Bind,

the single binding rule that compares a computed lane against a demanded one, applies only where a qualifier is written.

This result targets a number the paper must not report as evidence. A checker like this one is routinely validated with a corpus figure, "we ran the checker over N existing programs, and it rejected none of them", read as precision. Here that figure is true by construction. Existing programs state no demands, because the syntax did not exist when they were written. Zero rejections is guaranteed before anything runs, and the run would not even exercise the join, whose verdict goes unread without a demand. What is elsewhere a result is here a theorem, and it is why the evaluation's evidence is built from things that can fail instead: controls that must fire, an enumeration that must agree, and assertions that weaken a demand and require the verdict to move.

4.3 Blame

On violation, the blame rule walks back along the input, carrying the weakest lane, until it reaches a node whose class matches that lane. That node introduced the weakness itself rather than inheriting it, so it is provably a culprit, though not provably the nearest one. The checker reports both ends and the distance between them. Reporting only the demand site would invite the wrong repair, which would be a weakened demand.

Figure 4 shows the diagnostic. It fires on the program below, shown without its licence header and fixture directive; the figure counts lines of the file itself, header included, which is why the demand sits at line 21:

```
package app
func main() Float {
    x := indices(count: 100000.0)
    approx pre := sum(x * 0.25)
    replay demanded := sum(x + pre)
    return demanded
}
```

4.4 Healing

Introducing a name should not change a program's determinism:

```
s := sum(x)                  // named: one producer healed
replay e := sum(x*s)

replay e := sum(x * sum(x))  // inline: nothing to heal
```

Same program, same bits, one `:=` apart, and without healing the first form is rejected while the second is accepted. The first is in the fixture suite, and a control build that removes the healing pass must see it refused, which is the argument for the pass.

A demand's reach is decided by dataflow, not by where the text sits, and healing is what makes that true. The qualifier itself covers only its right-hand side. Healing reaches the named producers above it. A declared return lane covers nothing at all, a promise checked against the body rather than an ambient over it, so in a body that states no lane of its own, a fold reaches the fixed tree by healing or not at all.

That is healing's job. A declaration will often demand more than the producers above it promise, because upstream code carries no annotations for it. The checker could reject and make the programmer annotate every producer by hand. Instead it *heals*. Where a

```
reject_poison.lane:21:5: error: determinism lane violation: this value
computes approx where replay is demanded. An unpinned schedule cannot
be recovered downstream: the rounding already happened. Pin the producer
by naming it with a replay or exact qualifier, or weaken the demand
    21 |     replay demanded := sum(x + pre)
       |     ^^^^^^ the demand is stated here
                  ... the guarantee was lost 2 steps upstream:
    20 |     approx pre := sum(x * 0.25)
       |                   ~~~ introduced here
```

Figure 4: Blame, verbatim from the checker.

producer is *private to the demand*, with no live read-path out of the demanded cone, the checker rewrites its class up to what the demand needs, as if the demand had been written at the source. The pass is ordinary annotation inference, and the privacy condition is what keeps it sound: pinning a private producer changes nothing an outsider can see.

Whether a producer is private is decided over every path that reads it, and its direct consumers alone cannot decide it. One fixture, entire but for its licence header and directive:

```
package app
func main() Float {
    x := indices(count: 100000.0)
    partial := sum(x * 0.5)
    y := partial + 1.0
    replay demanded := sum(x * y)
    return demanded + y * 0.0
}
```

`partial` has one direct consumer, `y`, and `y` feeds the demand, so a test on direct consumers calls `partial` private and heals it. But the return also reads `y`, so pinning `partial` would change bits an outsider sees, and the checker refuses with blame on `partial`. The rule has to follow the reads, and a path is cut only where it reaches the demanded value itself: a change an outsider observes *through* the demand is a change the demand licenses. The fixture is therefore refused.

A return demand heals the same way, through a node of its own that wraps the returned expression, and the body's producers are private to that node when nothing outside the body reads them. Healing settles every producer the schedule can move; what it cannot reach is a value the program never fixes at all.

4.5 Grounding

A column read from a file can carry `replay` as legitimately as the literal `3.0`, because reordering is not its problem, and that is the danger. Whether the file holds the same bytes next run is a fact about a deployment, not about arithmetic, and the lane has no way to ask. Grounding is the judgement that asks it, whether the program fixes a value's bytes at all, a question the lattice alone cannot pose. A value the program does not fix is *ungrounded*, and everything that enters from outside the program enters that way: no rule inside the language can settle its bytes. Every demand over data from outside will meet this refusal, and the blessing that follows, a promise the programmer takes on in writing, is its way through.

`grounded` is one bit per node, and it is a conjunction where the lane is a join. A leaf is grounded when the program fixes its value. Literals and generated columns are; an external source is not, and neither is a call to a function whose return type carries the

ungrounded qualifier. Every other node is grounded exactly when all its reads are, including the carried edge.

Demand replay over a value from outside, and grounding refuses it, because the program cannot prove the input fixed. One fixture:

```
package app
func main() Float {
    x := source(count: 1000.0)
    replay energy := sum(x * 2.0)
    return energy
}
```

Everything the lane can see is faultless, yet the fixture is refused, because the program has no way to know what bytes x will hold next run. There are three ways out: vouch for the input with a blessing, take it as a parameter so the obligation becomes the caller's, or weaken the demand to approx. §4.6 demonstrates the first.

4.6 Blessing

Grounding's refusal is correct about the world, the bytes a deployment supplies, and unfair to a programmer who knows theirs does fix them. The unfairness is real, because the lane over such a value is genuinely replay: a fixed-tree fold is a fixed-tree fold whatever the bytes were, and the semantics holds the input equal across the pairs the class quantifies over. The checker computes that lane correctly and provides no way to state it, and the one spelling that compiles, approx, is a licence to reorder the programmer never meant to grant.

@bless closes that gap without weakening anything. It attaches to a binding whose value is a leaf the program does not fix, and it does *not* make that leaf grounded. The leaf stays ungrounded for the life of the program. What the blessing adds is a second way to be accepted at the declaration: a demand whose every ungrounded leaf is blessed is accepted.

The conditional form matters, because it turns a claim into a receipt. A bare @bless is an unverified human promise. A conditional @bless(...) states what must hold of the input, and the run refuses before any arithmetic when it does not, which turns the promise into something a machine can check. Which conditions a language offers is the implementer's choice; this artefact ships one, a checksum over the loaded column.

The refused fixture of §4.5, blessed:

```
package app
func main() Float {
    @bless(0x1234abcd) x := source(count: 1000.0)
    replay energy := sum(x * 2.0)
    return energy
}
```

Such a blessing is written on a binding and covers the leaf that binding names, and no other. The fixture is now accepted.

5 Determinism safety

So far a lane is a label the checker moves around, and no result says the label means anything about bits. Section 5 closes that gap: it defines which runs must agree (the semantics), states the two facts about arithmetic the proofs take on trust (the assumptions), and proves that an accepted program keeps its promise (both theorems). Reproducibility is a claim about *pairs* of runs, so the proofs walk two runs side by side. One special schedule, the serial reference, separates exact from replay: a replay value agrees with itself across every allowed run, and an exact value also agrees with the reference.

5.1 The schedule-indexed semantics

A promise that bits will not move is empty until you say what may move. We draw that line where the artefact can witness it, and a source of variation is inside the promise only if a check varies it and watches the bits.

Given that standard, let S be the checked schedule set: every combination of worker count and traversal order, the two settings the evaluation varies. A pair from S might be two workers under the fused traversal against eight workers under 7-wide tiles. S excludes exactly three things: the host compiler, the vector width, and the ABI version.

Then add one schedule no deployment ever runs, oracle, the serial reference fold under the fixed build, and write S+ for the enlarged set. By definition, oracle counts as its own schedule, even if some deployment schedule happens to compute the same bits. Write bits(e, s) for the bit pattern e computes under a schedule s from S+, with the same inputs for every schedule.

We do not say how bits is computed. The proof treats it as a black box, and Assumption 1 is the only fact about the arithmetic it may use. Theorem 1 is therefore *parametric*: it holds for any arithmetic that satisfies Assumption 1, so its content is the lattice argument, not a fact about floating point. Such a theorem would be about nothing if no real arithmetic satisfied its assumption, and the model evaluator of §3.5 is the witness that one does, in a separate development with an audit of its own.

Each class names the set of schedule pairs its bits must agree across, written agrees:

agrees(exact) is every pair from S+, agrees(replay) every pair from S, and agrees(approx) is empty.

Each set strictly contains the next, and that strict containment is what puts exact at the bottom: the bigger a class's agreement set, the stronger its promise, and the strongest sits lowest.

5.2 The two assumptions

The proofs take two facts about the arithmetic on trust, one over schedules and one over worlds.

Assumption 1. An operation must not be where two runs start to differ. Feed it operands that agree bit for bit, and its result must agree too, across every schedule pair its own class covers; an approx operation covers none, so it owes nothing.

Take any operation o and any schedule pair in agrees(class(o)). If the operands of o agree bitwise across the pair, the bits of o agree too. A recurrence's carried value counts as an operand.

This means a difference between two runs is never created out of nowhere: it is either inherited through an operand, or born inside an operation whose class permits it.

Assumption 2. The fact grounding relies on, and the same statement the other way round, the schedule now held fixed and the input bytes varied: *take any operation o other than an external source, and any pair of worlds at one schedule, a world being one assignment of bytes to every external input; if the operands of o agree*

bitwise across the pair, the bits of `o` agree too. It is strictly weaker, assuming nothing about either the schedules or the arithmetic.

This means the world has exactly one door into a program, the external source: every other operation computes from its operands and nothing else.

5.3 Theorem 1 (schedule agreement)

Run an accepted program twice, under any two schedules its demand covers, and the demanded bits agree. The formal statement carries its conditions:

(i) *`P` is well formed, in the sense that every operand of a node is an earlier node;*
(ii) *Assumption 1 holds of it;*
(iii) *`L` is the assignment the join sweep computes, so that each node's lane is the join of its own class with its inputs', a recurrence's carried value among them;*
(iv) *a schedule pair `(s, s')` is fixed;*
(v) *"carried agreement" holds: at every recurrence whose own lane demands agreement over `(s, s')`, the carried value the two runs read agrees.*

Then for every node `u` such that (vi) `(s, s')` lies in `agrees(L(u))`, the bits of `u` agree across the pair.

Carried agreement is not assumed for free. In the evaluator a recurrence runs strictly in index order, so the carried value a tick reads was settled by the tick before. That is what discharges premise (v). The proof itself cannot supply it, because the carried edge is the one edge that points forward, with nothing earlier to induct from.

Apply the theorem to the opening program, at two workers and at eight. Both schedules lie in `agrees(replay)`, so the theorem covers `b` and the demanded bits agree across the pair; at `a` nothing is promised, because `agrees(approx)` is empty, and the difference the return computes is exactly what the incident's merge once destroyed.

The consequence is the sentence a programmer relies on: declare a lane, be accepted, and the bits agree across every pair that lane covers. *If additionally (vii) `u` is accepted at a declared lane `d`, meaning `u` declares `d` and `L(u)` is at least as strong as it, then the bits of `u` agree across every pair in `agrees(d)`.* It follows from the theorem because a stronger computed lane agrees across everything the declared one promises.

Adding a demand to a working program changes its numbers, by design. Healing rewrites op classes before lowering, making the graph the theorem constrains the *healed* one, which may differ from the program as written. Strengthening a producer changes which kernel it lowers to. Monotone acceptance is a promise about acceptance, and none about the bits.

5.4 Theorem 2 (the poisoning law)

Once a value has reached `approx`, nothing downstream restores the guarantee. However strict the later operation, the bits it reads were already made under a free schedule, and no kernel recovers what a different order would have produced.

If `computed(u) = approx` and `u` is an input of `v`, then `computed(v) = approx`, whatever the operation class of `v`. The proof is one line: `join` is `max` on the chain and `approx` is its top.

Consequence. No per-call API is sufficient. A check at `v`'s call site inspects `v`'s operation and its operand values, neither of which records `u`'s schedule history.

5.5 What is proved, and what is not

Fourteen results are mechanised in Coq: seven from nothing at all, and the other seven from only the eight axioms that frame the semantics, the schedule set, its oracle, worlds and `bits`. Proved:

- both theorems, and the consequence for declarations;
- §4.1's result in both directions, its fixed-point form, its sweep bound;
- the chain's two strict containments;
- five results about grounding.

Not proved: Assumption 1 itself, and that debt remains. Nine of the table's eleven rows are proved of the model evaluator. The two remaining, parallel scatter and transcendentals, are unchecked.

6 Discharge: lowering obligations

Section 6 covers what a backend owes each class and what discharges it. An accepted program is a promise the backend must keep.

6.1 Where the type system's responsibility ends

A checker can prove which kernel an operation lowers to. It cannot prove what that kernel does with the schedule, an arithmetic fact typing cannot reach. Treating the two as one obligation lets a determinism guarantee rest on a claim no check tested. Here the obligations are discharged by the evaluator's own kernels and witnessed row by row in Table 3. Every row but the last is imposed by a class; the last is owed by the human who wrote the blessing (§4.6).

Table 3: Lowering obligations: what is owed, what discharges it, and what witnesses it.

Obligation	Lanes	Mechanism	Gate	Status
fixed reduction tree	`replay`	the evaluator's fixed-tree kernel: blocks, then block results, folded strict left to right	L4, L2, CC2	**checked**
oracle-order fold	`exact`	the evaluator's serial strict fold	L4, L2, CC4	**checked**
contraction pinned off	`replay`, `exact`	the build's `-std=c11`, under which this toolchain does not contract; no gate reads the emitted code	none	**unbuilt**
canonical parallel scatter	`replay`	none: the only scatter here is serial	none	**unbuilt**
fixed math library	`replay` across platforms	none	none	**unbuilt**
FTZ/DAZ and rounding mode	all	not pinned	none	**unbuilt**
blessed input unchanged	any	an emitted checksum guard, refusing the run before any arithmetic over the column	L5	**checked**

6.2 The operation picks the kernel

The computed lane of a value never chooses the kernel that produced it. The ambient a demand sets and the order dial pick the kernel at resolution; lanes are computed afterwards and read that choice, and nothing flows back. If a computed lane could feed the choice, one class would denote two bit patterns depending on what was asked of it, which inverts the guarantee.

A kernel's remaining questions, its grouping width and its accuracy against a correctly rounded reference, are error *magnitude*: a less accurate fold is just as reproducible as a more accurate one, and `replay` stays silent on accuracy. Runtime cost splits the same way: the lattice runs at compile time, a demand changes only which kernel a fold lowers to, and its price is the kernel's number, not the lattice's. This artefact's single-threaded evaluator cannot price it. The two vendors that do put it at 15 percent or more [24] and at 20 to 30 percent [32].

6.3 What a lane costs the optimiser

A compiler writer's first question would be: *if values carry these labels, how much of my optimiser do I lose?* Choosing schedules is an array compiler's whole job, and a lane is a constraint on the schedule.

The general answer is one sentence.

> A lane is a property of a program, not of a transformation. Every transformation produces a new program; the join is recomputed on that program; and a transformation is lane-safe exactly when every surviving declaration is still discharged.

For example, fusion is lane-preserving exactly while the fold is not the fusion *sink*, the node everything fuses into. Each output element stays the same expression over the same inputs, no grouping moves, and map-into-map fusion, tiling and any dependence-preserving reordering are free where array compilers do most of their work. By contrast, fusing a map *into* a reduce does move the grouping, because the fused loop becomes the fold's loop and is parallelised as one. Six programs at five traversals and four worker counts show it: a value under a demand holds one bit pattern across all twenty cells, one under none up to four.

Reduction splitting is what the lane forbids. Halide's `rfactor` splits an associative reduction into partials under an associativity proved in the reals [39], and a value's bits do not obey them; a build that makes the mistake in the form it would really take, a tiled traversal accumulating per tile, is refused.

The class must also survive value numbering, and we learnt that from an incident. Two reductions over the same column, one demanded `replay` and one not, are structurally identical and *must not* merge, because they have different lowering contracts. The compiler this work grew from interned nodes *before* lanes were stamped, so the twins merged and were re-stamped with the stronger lane, silently pinning a cone the program did not ask to pin. We now put the contract in the interning key, and control CC5 removes it. The normal build lowers the twins to two kernels, `fold_free` and `fold_replay`; the CC5 build lowers them to `fold_free` alone, and the `replay` contract is gone with no diagnostic. A merge that changes no value can still lose a schedule contract. Spark states the discipline for source nondeterminism. We found none stating it for the floating-point schedule class.

7 Evaluation

What is evaluated, and by which figure. Three strands. The checker is compared against a path enumeration, 58,245 node answers over random graphs and 47,738 over front-end-built ones, with zero mismatches (§7.1). Enforcement is exercised by 8,000 metamorphic assertions over three properties, one of which weakens every demand (§7.1). Finally, the evidence itself is audited: nineteen must-fail controls, each watched as it fires (§7.3), and an inquiry into what each passing check could have caught, with §7.2 reporting the ten occasions on which a pass had proved nothing.

The assertion counts are machine-independent because every generator is a pure function of `(seed, n, i)`. The evaluator itself is single-threaded, and a worker count selects the chunking, so every schedule in `S` is exercised as the bracketing it induces, never by concurrent execution. Reproduce with `./reproduce.sh` and `./controls.sh`; every check and its command is tabulated in the artefact (Appendix A).

Three words, then the objection they must answer. A *check* is one of the artefact's numbered test programs, which a reader runs. A *control* is a break staged on purpose for a check to catch, a deliberate fault in the sense mutation testing gives the word [14]. A *receipt* is the status line a check prints, which is what a claim cites as its witness. The objection first: a check is another claim about the code, and a claim can rot like any other. A reader is therefore owed the reason this suite is not simply more documentation. Three things stop checks from needing checks of their own. Every control must fire, and a control that does not means its check is decorative and the run fails. §7.2 asks of every passing check what it could have caught. And two pieces of evidence are not check-shaped at all: §7.1's path enumeration and §5.5's mechanisation.

7.1 The join against exhaustive enumeration

Because monotone acceptance (§4.2) rules out the conventional corpus figure, we compute every node's class twice, by two methods that share as little as possible. That is differential testing [26], with the enumeration as the reference. The implementation folds `join(op_class, inputs…)` once per node in index order. The enumeration walks *every path* from every source to the node, takes the weakest class on each path, then the weakest across paths. It is exponential and shares neither the sweep's traversal nor its argument about dependency order.

Both call the same two primitives, so the enumeration cannot cover the join itself; since join is idempotent `max`, the weakest along each path and then across paths equals the recursive join, and the check is that the single indexed sweep visits the right nodes in the right order. Over 2,000 random DAGs of 8 to 50 nodes, 58,245 node answers agreed, with zero mismatches, and no node exceeded the path cap of 200,000 enumerated paths.

And the same enumeration again, over programs the compiler built. L6 generates well-typed *source*, drives the real front end over it, and runs this same exhaustive enumeration on the resulting DAG: 47,738 node answers, zero mismatches. The generator declares no functions, and the inlining stays with the fixtures.

In addition, the metamorphic properties, relations between paired runs in the metamorphic-testing sense [11], add 8,000 assertions. Weakening every demand to `approx` always yields zero violations, and weakening never turns accept into reject (M1). The violation count never decreases from `replay` to `exact` (M2). And checking twice changes nothing (M3).

7.2 The check audit: ten incapable of failing

Since §4.2 rules out the usual no-regressions figure, we report the opposite one: every occasion on which a check of ours was green over evidence that could not have shown it the thing it existed to catch. There are ten, found across the twelve checks the artefact runs and the measurements and controls behind them, in two classes: three are defects in a *measurement* whose subject was sound, and seven are defects in what we pointed a check at. Four bear directly on the type system and follow; the rest are catalogued with the artefact (Appendix A).

1. The invariance check measured the wrong graph. §5.3 constrains the *healed* graph, and the check evaluated the graph as written. It went unseen while every corpus program annotated the fold directly, where no producer needs healing. The first program that needed a heal reported four bit patterns where the accepted program has one.

2. A value test blind to a merge. §6.3's incident is two structurally identical reductions with different contracts collapsing into one, and no assertion over values can see it. The check needs a fact about the merge, and the check that catches it asserts over the emitted kernels instead.

3. Every schedule-varying check asked whether a value agreed with itself. Varying the schedule and comparing bits is a question a deterministic and consistently wrong kernel answers correctly. No check in the suite compared a value to the serial reference that `exact` names, which is the content of that class. L4's reference comparison is the repair, and control CC4 is a build that discharges one lane with two different bit patterns while leaving every invariance assertion green.

4. The round-trip check could not see the healing it names. L6 heals each generated program, then compares every node's lane against the path enumeration. Both sides read the healed graph, and deleting the pass changed no verdict. The repair also reads the lanes before the heal, and requires the corpus to move some: 371 producers strengthened, and 2,809 of the 47,738 answers changed.

All ten were green when found, and none was found by a test failing.

7.3 The checker's must-fail controls

The controls are nineteen in all, numbered CC1 to CC19. Seventeen break a build of the checker, the evaluator, or the mechanised proof. CC6 and CC8, a malformed program and the kill switch, an environment variable that disables the checker, assert directly on the shipped binary.

7.4 Where CCCL's guarantee stops

A gap can have a shape. The composition question is not one CCCL answers wrongly; it is one the library gives a caller nowhere to ask, because no determinism level outlives the call that stated it. An absence argued in prose is weak evidence, and check L1 runs it as an experiment, inside the vendor's compiler, where every other experiment here runs in ours.

Three programs, compiled with `nvcc` against the CCCL headers. Two of them state something the library can decide locally and must reject: an unsatisfiable triple of determinism level, operator and element type, and a determinism property passed without `require`. Both fail to compile, each with its own named diagnostic. The third takes the result of a `not_guaranteed` reduction and feeds it to a `run_to_run` one. It compiles without a word. The second reduction runs over one element, which no schedule can reorder. The case therefore rests entirely on what the result carries, and it carries nothing for the second call to check. The two rejections are what make the third case mean anything.

The same shape in lanelang is a fixture: a free fold passed to a parameter declared `replay`, read elsewhere by the caller so healing cannot strengthen it, and refused with blame at both ends. A private fold would have been healed instead. Either way the mix is decided, and CCCL compiles it.

7.5 The opening case, replayed

The paper opened on 10,179 reruns, and 31% changed trees. The authors' stated mechanism is that parallel reductions re-associate as a function of the core count, and the drifted likelihoods then reach the comparisons that steer the search [38]. A drift in the ninth digit is harmless until a branch reads it. Check L4 replays that shape at this artefact's scale, as a free fold feeding a recorded-value test:

```
package app
func main() Float {
    x := indices(count: 100000.0)
    approx s := sum(x * 0.5)
    picked := choose(when: s - 0.0042511812704049134,
                     then: x * 0.0 + 3.0,
                     else: x * 0.0 + 5.0)
    total := isum(picked)
    return total
}
```

The constant is the sum a serial run of this program records, which makes the program a regression test against a golden value, the form in which such drift reaches practice (`indices` fills ill-conditioned cancellation pairs from a fixed seed, not `0, 1, 2`, and a non-zero mask picks `then`). Across the twenty schedule cells, the free version returns both arms, because every worker count but one drifts off the recorded bits, in the ninth significant digit. Pin the producer, and each class holds one answer everywhere. Demand `replay` of the result while the mask stays free, and the checker instead refuses with two-span blame naming the fold (a refused fixture of L3). The regression test that would have broken under a different worker count is instead refused before it ever runs.

8 Related work

Section 8 covers the graded systems, the order-fixing family, and the security lineage this design borrows from; the ordering mechanisms themselves are §2's census.

Determinism is already checked at plan time. Flink can reject a query whose column "can not satisfy the determinism requirement" [41], and Spark's Catalyst computes a per-expression `deterministic` property as a meet over the dataflow DAG, whose

`semanticEquals` refuses to merge nondeterministic twins: §6.3's lesson, already shipping [42]. Determinism checking with blame over a dataflow plan is *not new*. Their property is binary, over source nondeterminism, and imposed by the engine.

Graded determinism as a checked type. Mercury has had compiler-checked determinism declarations since the 1990s [37], and the graded systems after it each grade a different subject [9, 13, 16, 22, 29, 30, 36], while graded modal types and coeffects [34] grade a variable's *use*, where this work grades a value's whole *history*, joined over the dependency graph. The machinery is claimed from none of them, and the lattice frame itself is Denning's [15].

The discharge into lowering is not novel either: EnerJ lowers its qualifiers into approximate storage and functional units [36]. Nor is the three-level vocabulary ours, since Veit's kernel contracts publish `BITWISE | RUN_TO_RUN | NONE` for ML kernels [49], declared over an operation class and checked by repeated invocation on one machine. In contrast, a lane is carried on a value. That work reaches the join and names it as future work in its own voice: what two contracted kernels compose to is, in general, not the conjunction of their contracts. It is the closest thing to a counterexample to the novelty claim, and it is the strongest evidence for it.

Properties on array types. SaC's type patterns carry shape constraints per signature, re-established at every application and never carried on the value [1].

Licences inside a value grammar. Icing comes nearest, putting the optimisation licence *inside the value*'s grammar [7]; verified compilation of floating-point programs continues that line [8]. Icing is strictly sequential, quantifying over which compiler rewrites fired, whereas we quantify over which runtime schedule ran.

Fixing the order. The family that pins an implementation is conceded entirely. It is ancient, the fixed shape arriving independently at least four times between 1964 and 1970 [5, 23, 25, 35], and it is still being improved, from numpy's partial pairwise summation, applied conditionally [47], to oneTBB's deterministic reduce [48] to ReproRed [38]. Sixty years of implementations pin the order; what none of them publishes is the checked contract.

Not fixing the order. Where the family above pins a schedule, Demmel and Nguyen's reproducible summation and ReproBLAS make the sum order-*independent* so that no schedule can disagree [2]. Although it looks like an `exact` mechanism, it is an alternative discharge of `replay`: a function of the data alone that still differs from the serial fold's answer.

The nearest structural precedent, in security rather than numerics. FaCT [10] grades a per-value secrecy label, joins it along the dependency path, checks it against a declared one, rejects where a linter would advise, and discharges acceptance into a constant-time lowering obligation: every element of §2.1, in another subject, in 2019. What differs is the quantifier.

Their pair of runs holds the program fixed and varies the secret, observing a leakage trace [3, 4, 6], whereas ours holds the input fixed and varies the schedule, observing the bits. Both are 2-safety properties, each blind to the other's subject.

9 Limitations

One limit is the design's reach as shipped: Table 2 leaves transcendentals at `approx`, so no fold fed by an `exp` could demand `replay`. Every other limit here is a limit of the evidence, not of the lattice. *Each receipt is one machine, one toolchain*, so no claim is made across devices, GPU vendors or CPU ISAs; the evaluator's kernels are the only backend anything here witnesses.

In addition, FTZ/DAZ (flush-to-zero and denormals-are-zero) and the rounding mode are not pinned, although an exact replay must run under the same flags, and no check covers them.

A demand over external data rests on a human (§4.6), and an input that varies by nature, a live stream say, is genuinely ungrounded and stays refused.

Determinism safety here is conditional, on Assumption 1's operation-class table and on the fixed build, and §5.5 states what remains unproved. The language is a demonstration, not a production compiler: pure, first-order, non-recursive, no transcendentals, one numeric type, and no type checking beyond the two types `Float` and `Column[Float]`.

The conditional, the simplification most likely to look structural, is the one we can show. `choose` takes both arms as operands, and the mask joins as an ordinary one.

`replay` constrains the schedule only. The runtime gap between a pinned kernel and the free one is the kernel's number, and §6.2 gives the field's pricing of it. What can be stated of the engineering cost is the surface adoption touched in the production compiler the incident comes from: an annotation on the syntax tree, a module-wide ambient lane, one integer on the IR node, three fields in the value-numbering key, one join pass, one heal pass, and a registry of sixty-nine rows, with no change to that compiler's type checker. The adoption found eight lane defects, two of them unsound acceptances.

10 Conclusion

A demand carried on the value and joined over its history is enough to make reproducibility implementable with a type. The schedule is one subject amongst many, and a neighbour of this work already grades another. The Determinism Checker carries a three-point class of the same shape over the iteration order of an unordered collection [30]. Therefore, with this design, you can keep the checker's implementation and swap out the operation-class table, to implement this for other sources of nondeterminism.

A The artefact

The artefact is at https://github.com/politepixels/replay-lattice-paper and archived at https://doi.org/10.5281/zenodo.22333026.

References

[1] Jordy Aaldering, Sven-Bodo Scholz, and Bernard van Gastel. 2023. Type Patterns: Pattern Matching on Shape-Carrying Array Types. In *Implementation and Application of Functional Languages (IFL)*. doi:10.1145/3652561.3652572

[2] Willow Ahrens, Hong Diep Nguyen, and James Demmel. 2016. ReproBLAS: Reproducible Basic Linear Algebra Sub-programs. https://bebop.cs.berkeley.edu/reproblas/.

[3] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub. 2017. Jasmin: High-Assurance and High-Speed Cryptography. In *Computer and Communications Security (CCS)*. 1807–1823. doi:10.1145/3133956.3134078

[4] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, François Dupressoir, and Michael Emmi. 2016. Verifying Constant-Time Implementations. In *USENIX Security Symposium*. 53–70.

[5] Ivo Babuška. 1969. Numerical Stability in Mathematical Analysis. In *Proceedings of the IFIP Congress 68*. North-Holland, 11–23.

[6] Gilles Barthe, Sandrine Blazy, Benjamin Grégoire, Rémi Hutin, Vincent Laporte, David Pichardie, and Alix Trubiani. 2020. Formal Verification of a Constant-Time Preserving C Compiler. *Proceedings of the ACM on Programming Languages* 4, POPL (2020), 7:1–7:30. doi:10.1145/3371075

[7] Heiko Becker, Eva Darulova, Magnus O. Myreen, and Zachary Tatlock. 2019. Icing: Supporting Fast-Math Style Optimizations in a Verified Compiler. In *Computer Aided Verification (CAV) (LNCS, Vol. 11561)*. 155–173. doi:10.1007/978-3-030-25543-5_10

[8] Heiko Becker, Robert Rabe, Eva Darulova, Magnus O. Myreen, Zachary Tatlock, and Ramana Kumar. 2022. Verified Compilation and Optimization of Floating-Point Programs in CakeML. In *European Conference on Object-Oriented Programming (ECOOP) (LIPIcs, Vol. 222)*. 1:1–1:28. doi:10.4230/LIPIcs.ECOOP.2022.1

[9] Robert L. Bocchino, Vikram S. Adve, Danny Dig, Sarita V. Adve, Stephen Heumann, Rakesh Komuravelli, Jeffrey Overbey, Patrick Simmons, Hyojin Sung, and Mohsen Vakilian. 2009. A Type and Effect System for Deterministic Parallel Java. In *OOPSLA*. 97–116. doi:10.1145/1640089.1640097

[10] Sunjay Cauligi, Gary Soeller, Brian Johannesmeyer, Fraser Brown, Riad S. Wahby, John Renner, Benjamin Grégoire, Gilles Barthe, Ranjit Jhala, and Deian Stefan. 2019. FaCT: A DSL for Timing-Sensitive Computation. In *Programming Language Design and Implementation (PLDI)*. 96–111. doi:10.1145/3314221.3314605

[11] T. Y. Chen, S. C. Cheung, and S. M. Yiu. 1998. *Metamorphic Testing: A New Approach for Generating Next Test Cases*. Technical Report HKUST-CS98-01. Department of Computer Science, The Hong Kong University of Science and Technology.

[12] Michael R. Clarkson and Fred B. Schneider. 2010. Hyperproperties. *Journal of Computer Security* 18, 6 (2010), 1157–1210. doi:10.3233/JCS-2009-0393

[13] Eva Darulova and Viktor Kuncak. 2014. Sound Compilation of Reals. In *Principles of Programming Languages (POPL)*. 235–248. doi:10.1145/2535838.2535874

[14] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer* 11, 4 (1978), 34–41. doi:10.1109/C-M.1978.218136

[15] Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236–243. doi:10.1145/360051.360056

[16] Joseph Devietti, Dan Grossman, and Luis Ceze. 2012. The Case For Merging Execution- and Language-level Determinism with MELD. In *Workshop on Determinism and Correctness in Parallel Programming (WoDet)*. https://homes.cs.washington.edu/~djg/papers/wodet_meld.pdf

[17] Horace He and Thinking Machines Lab. 2025. Defeating Nondeterminism in LLM Inference. Thinking Machines Lab: Connectionism. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/. doi:10.64434/tml.20250910

[18] Troels Henriksen. 2026. Futhark prelude: `reduce`. `prelude/soacs.fut`.

[19] IEEE. 2019. IEEE Standard for Floating-Point Arithmetic. IEEE Std 754.

[20] William Kahan. 1965. Pracniques: Further Remarks on Reducing Truncation Errors. *Commun. ACM* 8, 1 (1965), 40. doi:10.1145/363707.363723

[21] Ulrich Kulisch and Van Snyder. 2011. The Exact Dot Product as Basic Tool for Long Interval Arithmetic. *Computing* 91 (2011), 307–313. doi:10.1007/s00607-010-0127-7

[22] Lindsey Kuper, Aaron Turon, Neelakantan R. Krishnaswami, and Ryan R. Newton. 2014. Freeze After Writing: Quasi-Deterministic Parallel Programming with LVars. In *Principles of Programming Languages (POPL)*. doi:10.1145/2535838.2535842

[23] Peter Linz. 1970. Accurate Floating-Point Summation. *Commun. ACM* 13, 6 (1970), 361–362. doi:10.1145/362384.362498

[24] Livermore Software Technology. 2021. `*CONTROL_PARALLEL` and `*CONTROL_MPP_IO_LSTC_REDUCE`. LS-DYNA Keyword User's Manual, Volume I, R13, pp. 12-404 and 12-425 to 12-427.

[25] Daniel D. McCracken and William S. Dorn. 1964. *Numerical Methods and Fortran Programming*. Wiley.

[26] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.

[27] Message Passing Interface Forum. 2023. MPI: A Message-Passing Interface Standard, Version 4.1. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf.

[28] Robin Milner. 1978. A Theory of Type Polymorphism in Programming. *J. Comput. System Sci.* 17, 3 (1978), 348–375. doi:10.1016/0022-0000(78)90014-4

[29] Alexandre Moine, Sam Westrick, and Joseph Tassarotti. 2026. All for One and One for All: Program Logics for Exploiting Internal Determinism in Parallel Programs. *Proceedings of the ACM on Programming Languages* 10, POPL, Article 26 (2026). doi:10.1145/3776668

[30] Rashmi Mudduluru, Jason Waataja, Suzanne Millstein, and Michael D. Ernst. 2021. Verifying Determinism in Sequential Programs. In *International Conference on Software Engineering (ICSE)*. 37–49. doi:10.1109/ICSE43902.2021.00017

[31] Arnold Neumaier. 1974. Rundungsfehleranalyse einiger Verfahren zur Summation endlicher Summen. *ZAMM* 54 (1974), 39–51. doi:10.1002/zamm.19740540106

[32] NVIDIA. 2026. Controlling Floating-Point Determinism in NVIDIA CCCL. NVIDIA Developer Blog and `github.com/NVIDIA/cccl`.

[33] OpenMP Architecture Review Board. 2021. OpenMP Application Programming Interface, Version 5.2, Section 5.5.6, Reduction Scoping Clauses.

[34] Dominic Orchard, Vilem-Benjamin Liepelt, and Harley Eades III. 2019. Quantitative Program Reasoning with Graded Modal Types. *Proceedings of the ACM on Programming Languages* 3, ICFP, Article 110 (2019), 30 pages. doi:10.1145/3341714

[35] Heinz Rutishauser. 1967. Description of ALGOL 60, Chapter 5: Miscellaneous Applications. In *Handbook for Automatic Computation, Vol. 1a*. Springer, 95–143. doi:10.1007/978-3-642-86934-1_5

[36] Adrian Sampson, Werner Dietl, Emily Fortuna, Danushen Gnanapragasam, Luis Ceze, and Dan Grossman. 2011. EnerJ: Approximate Data Types for Safe and General Low-Power Computation. In *Programming Language Design and Implementation (PLDI)*. doi:10.1145/1993498.1993518

[37] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. 1996. The execution algorithm of Mercury, an efficient purely declarative logic programming language. *Journal of Logic Programming* 29, 1–3 (1996), 17–64.

[38] Christoph Stelz, Lukas Hübner, and Alexandros Stamatakis. 2026. Bit-reproducible parallel phylogenetic tree inference. *Bioinformatics* 42, 2 (2026), btag044. doi:10.1093/bioinformatics/btag044

[39] Patricia Suriana, Andrew Adams, and Shoaib Kamil. 2017. Parallel Associative Reductions in Halide. In *Code Generation and Optimization (CGO)*. doi:10.1109/CGO.2017.7863747

[40] Tachio Terauchi. 2008. A Type System for Observational Determinism. In *IEEE Computer Security Foundations Symposium (CSF)*.

[41] The Apache Flink Project. 2022. Determinism in Continuous Queries. Flink documentation, Table API. https://nightlies.apache.org/flink/flink-docs-release-1.20/docs/dev/table/concepts/determinism/

[42] The Apache Spark Project. 2026. `Expression.deterministic` and `semanticEquals`. Spark source, `catalyst` expressions, file `Expression.scala`.

[43] The Khronos Group. 2023. The OpenGL Shading Language, Version 4.60.8. OpenGL Registry. https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.html

[44] The Khronos Group. 2024. SPV_KHR_float_controls2. SPIR-V extension registry. https://github.khronos.org/SPIRV-Registry/extensions/KHR/SPV_KHR_float_controls2.html

[45] The LLVM Project. 2026. Fast-math flags and EarlyCSE attribute intersection. LLVM source, `lib/Transforms/Scalar/EarlyCSE.cpp`.

[46] The MLIR Project. 2026. 'arith' Dialect: FastMathFlagsAttr. MLIR documentation, `mlir.llvm.org/docs/Dialects/ArithOps/`.

[47] The NumPy Project. 2026. numpy.sum. NumPy v2.5 Manual, reference documentation. https://numpy.org/doc/stable/reference/generated/numpy.sum.html

[48] UXL Foundation. 2026. `parallel_deterministic_reduce`. oneTBB specification, `doc/main/specification/source/algorithms/functions/parallel_deterministic_reduce_func.rst`; published as oneAPI Specification 1.4-provisional-rev-1, oneTBB element.

[49] Cooper Veit. 2026. Kernel Contracts: A Specification Language for ML Kernel Correctness Across Heterogeneous Silicon. arXiv preprint. arXiv:2604.22032 [cs.LG]

[50] Dennis Volpano, Cynthia Irvine, and Geoffrey Smith. 1996. A Sound Type System for Secure Flow Analysis. *Journal of Computer Security* 4, 2-3 (1996), 167–187. doi:10.3233/JCS-1996-42-304

[51] Steve Zdancewic and Andrew C. Myers. 2003. Observational Determinism for Concurrent Program Security. In *Computer Security Foundations Workshop (CSFW)*. 29–43. doi:10.1109/CSFW.2003.1212703